EECS 581 – Team 15
Project 3
October 24th, 2026

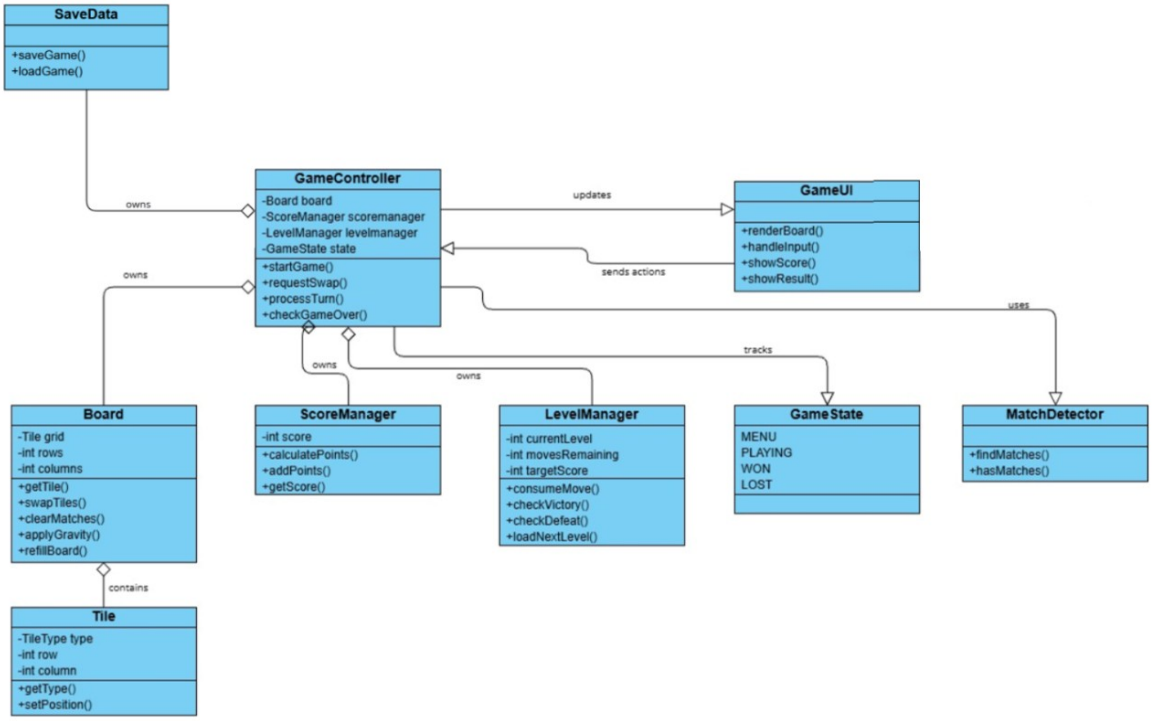
Initial Architecture

Emilia Davis, Ryan Graham, Daniel Harris, Eliza Malyshev, Zachary Mccauley, Adria Mongar, Jett Viduya

Synopsis

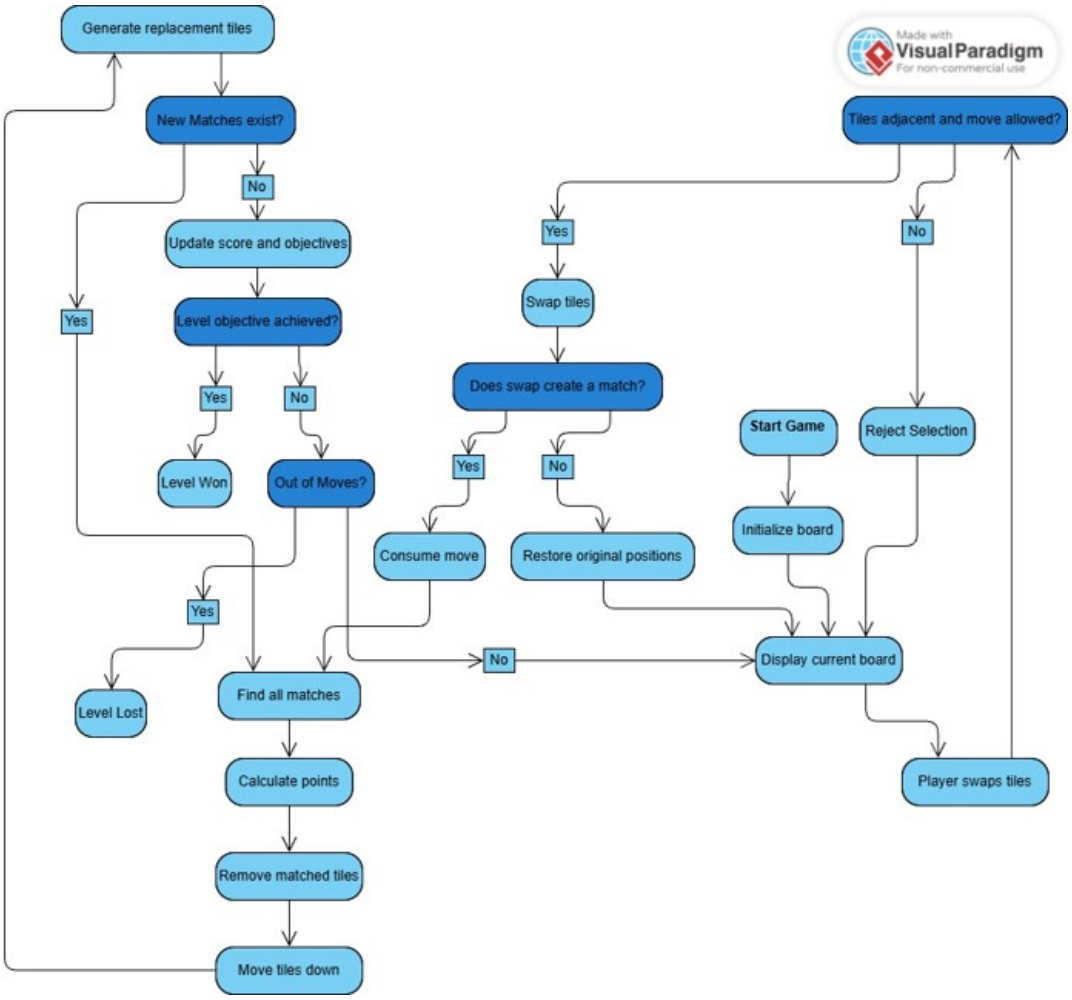
The goal of this project is to develop a game with similar game play to Candy Crush, with distinct levels and mechanics.

Architecture Description



Main Gameplay

The main game play consists of some fixed-size grid, where tiles fall in from the top and matches can be made to clear tiles. Each grid space has an object representing it, with information about its contents and indicating which grid space (if any) tile contents should fall from if it is empty. Whenever a move is made, a method on each grid space can be called to determine if it is part of a match. If so, the tiles can be cleared out, a powerup can be placed into the tile, and tiles can naturally fall from there.



Inputs

Inputs will be handled using a separate *InputManager* class. When the game initially loads, an *InputManager* instance will be created, and will find every possible input that is necessary to run the game. When various things in the game depend on inputs, they can find the inputs through the *InputManager*, and enroll any functions to those inputs.

In addition, the *InputManager* will have methods to enable and disable sets of inputs when it makes sense to do so (such as having menu inputs enable when a menu is brought up and vice versa).

The game will support keyboard and mouse inputs, and if time permits, may support others forms of input as well (such as controller inputs).

Menus

The game uses various menus, including but not necessarily limited to:

- Level selection
- Settings
- Pause menu
- Title screen

With the exception of the pause menu, all menus exist in their own separate scene in Unity; when the menu is entered, all previous information is unloaded, and when the menu is exited, the menu is removed from memory. Menus contain UI elements included in the base Unity game engine, including (primarily) buttons, sliders, text input boxes, etc. Buttons can have functions tied to them which are called when the button is pressed, and other user inputs can have their state read when necessary.

The pause menu is slightly more complicated in that it will have to be loaded on top of the game play Scene, and all game play elements must be paused while the pause menu is open. Time should not pass in the game play scene, and inputs should not be processed for the game play scene. The Unity game engine provides simple ways of doing this, by setting the time scale to zero, and having the game play inputs be on a separate input map from the menu inputs, and enabling/disabling both when relevant.

Saved Data

As the game is played, various data will need to be kept track of in-between sessions. This data includes:

- Level completion status
- Level high scores
- Level progress (if the user quits mid-level)
- Any settings the user has set

To do this, we will define an abstract *SaveData* class, which has methods for saving and loading data. Information of similar nature will be grouped together in different subclasses of *SaveData* and stored in separate files. *SaveData* uses a static method to load files, and will throw an exception if the file does not exist, or if the file contents can't be parsed into an instance of *SaveData*.

Scene Management

Scenes in Unity are sets of objects that can be loaded all at once. Each menu will have its own scene, as well as the main game play scene. Generally, whenever a scene is loaded, all previous game objects will be unloaded. However, there are a few exceptions: when the pause menu is loaded, the main game play loop will remain loaded, and some objects will be set not not be destroyed when a new scene is loaded (particularly those integral to running the program, such as the *InputManager*). A special class will be accessible and will have methods to load (and unload) scenes as necessary.

